

CampusCore API

Next-Generation College Management System

1. Executive Summary & Objective

The **CampusCore API** is a high-performance, backend-only College Management System designed to centralize and automate institutional data operations. By eliminating fragmented spreadsheet tracking, it provides a unified, secure database for managing a university's core entities: academic departments, faculty, students, and library resources.

Primary Objective: To build a robust RESTful API using modern Python frameworks and a NoSQL database, demonstrating full CRUD (Create, Read, Update, Delete) capabilities alongside complex database relationship mappings (1:1, 1:N, N:1, N:M) without the reliance on a traditional frontend web interface.

Technical Focus: This project strictly enforces database integrity, high-speed asynchronous data processing, and built-in API documentation via Swagger UI, ensuring an industry-standard backend architecture.

2. System Architecture & Entity Mapping

The system models four critical real-world collegiate entities using MongoDB's document-based structure and Reference ObjectIDs to map relationships cleanly:

1. Departments Collection

The central hub for academic branches.

- **Relationship (1:1):** Maps exactly one Head of Department (HOD) via a `hod_id` reference to the Staff collection.
- **Relationship (1:N):** Acts as the parent branch for multiple Students and Staff members.

2. Staff Collection

Tracks faculty profiles, designations, and roles.

- **Relationship (N:1):** Connects to a parent Department via a department_id reference.
- **Business Logic:** Includes validation to ensure only staff with a "Professor" designation can be assigned as a Department HOD.

3. Students Collection

Maintains student academic enrollment and library borrowing status.

- **Relationship (N:1):** Connects to a parent Department via a department_id reference.
- **Relationship (N:M):** Maintains an array of borrowed_book_ids to track active library loans.

4. Books Collection (Library System)

Manages the university's library inventory and active circulation.

- **Relationship (N:M):** Maintains an array of issued_to_student_ids.
- **Business Logic:** Implements a smart constraint that restricts students to a maximum of 3 active book loans at any given time.

3. Tools and Technologies Stack

To fulfill the strict backend-only prerequisite while ensuring a modern, scalable application, CampusCore utilizes the following technology stack:

Programming Language	Python 3.x – Chosen for its rapid development capabilities, readable syntax, and powerful ecosystem for backend data manipulation.
Backend Framework	

	FastAPI – A highly performant, asynchronous web framework for building APIs. It validates data types automatically and natively supports async/await patterns for database queries.
Database (NoSQL)	MongoDB – Used to store, manipulate, and map data entities. It allows for flexible schema design and uses Reference ObjectIDs to simulate traditional table joins efficiently.
Database Driver	Motor (AsyncIOMotorClient) – The official asynchronous Python driver for MongoDB, allowing the FastAPI server to handle database operations without blocking concurrent requests.
Data Validation	Pydantic – Integrated with FastAPI to enforce strict schema rules, ensuring that data entering the database is correctly formatted before insertion.
API Testing & UI	Swagger UI (Built-in) / Postman – FastAPI automatically generates an interactive Swagger documentation page, fulfilling the requirement for a frontend-free testing environment.